

Digital Media II – 008

Final Project

Boardventure: Spin, Play, Win

Game Title: BoardRush

BoardCraft Creations

Jolomi Eddi

and

Ruhani Jindal

University of Winnipeg Collegiate

Final Project Report

1. Introduction and Motivation

This project was inspired by the game of life and other board games we enjoy playing. The motivation behind this game was to create a fun online game for people who feel bored with traditional board games. It evolved into becoming a game for people who want to play a board game without physical games with them. Our game aimed to provide a fun experience that is accessible to everyone. The game is at a good level for us in terms of the coding and is aesthetically pleasing along with being functional.

2. Background and Related Works

a. First Related Work: Game of Life

The Game of Life serves as a strong inspiration for this project due to its structured board game mechanics, player progression system, and emphasis on chance-based movement combined with strategic decision-making. Like *The Game of Life*, our game involves players taking turns, spinning a wheel to determine movement, and encountering spaces that impact their journey.

One key similarity is the presence of **event-based spaces**. In *The Game of Life*, players land on spaces that trigger life events such as career changes, financial gains or losses, and other milestone moments. Similarly, our game features **action card spaces**, where players engage in creative actions for an element of surprise in each turn. These action spaces introduce unpredictability and variation, making each playthrough unique.

Another important parallel is the **currency or point-based progression system**. In *The Game of Life*, financial success is a major factor in determining the winner. Likewise, in our game, players will earn points or in-game currency based on their action cards. Rather than simply racing to the finish, success depends on how well players perform throughout the game.

Additionally, *The Game of Life* emphasizes a **blend of luck and player choice**. While movement is determined by chance (via the spinner), the outcomes of life events and financial decisions impact the final standings. Our game adopts a similar structure, where luck influences movement, but action cards determine overall success.

By drawing inspiration from *The Game of Life*'s turn-based structure, chance-based movement, and event-driven spaces, our game aims to provide an engaging and interactive experience while introducing dynamic action cards and animations to set it apart.

b. Second Related Work/Inspiration: Monopoly

Monopoly serves as a key inspiration for this project due to its structured board game mechanics, player progression system, and strategic decision-making elements. Like Monopoly, our game involves players taking turns, moving across a game board based on a randomized movement

mechanic (in our case, a spinning wheel rather than dice), and encountering specific spaces that impact their progress.

One of the most significant similarities is the presence of **action spaces**—in Monopoly, players may land on properties, Chance or Community Chest cards, or other special tiles that require them to take an action. Similarly, in our game, action cards include actions that players must complete, adding an element of unpredictability and engagement.

Another parallel is the **currency system**. In Monopoly, players accumulate and manage money throughout the game, which determines their success. In our game, players will earn in-game currency or points, influencing the final outcome. This mechanic ensures that the winner is not simply the first player to reach the final tile but rather the one who has earned the most through skill and strategy.

Additionally, Monopoly's **balance of luck and strategy** is reflected in our design. While movement is randomized, strategic decisions will significantly impact their success. This blend of chance and skill makes for an engaging and competitive gameplay experience.

By drawing inspiration from Monopoly's turn-based mechanics, action spaces, and money-based scoring system, our game seeks to create an exciting and interactive experience while introducing fresh elements such as animated movement and action cards to differentiate itself.

3. Problem Statement

Our goal was to design and develop a fun, engaging board game that targets individuals aged 7-19. We aim to deliver a game that is simple to understand yet filled with enjoyable surprises, challenges, and interactive elements to keep players entertained during their free time.

4. Solution Strategy and Implementation

For the game to feel complete and engaging, we implemented several key mechanics that work together to enhance the overall experience. Each part was designed to make the gameplay more interactive, rewarding, and enjoyable. The key sections of the implementation of our plan were:

a. Player Movement Animation

To enhance the gameplay experience, an animation system has been implemented to visually represent player movement across the board. When a player spins the wheel, their game piece moves step-by-step across the appropriate number of spaces, ensuring a smooth and immersive transition.

b. Action Cards

Certain spaces on the board will feature action cards with unique actions. When a player lands on one of these spaces, they must complete the associated action, often automated, before proceeding. These actions serve as a core gameplay mechanic, adding variety and strategy to the experience.

c. Currency & Score Tracking System

A point or currency system has been implemented to reward players based on their action cards. This system allows players to accumulate points, which plays a key role in determining the game's winner.

d. Game Conclusion & Winner Determination

The game ends when all players reach the final tile on the board. At this stage, the player with the highest accumulated score or currency will be declared the winner. This scoring system ensures that success is based on both progress and performance throughout the game.

5. Experimental Framework

Our approach to this project evolved quite a bit from our original plan. Initially, we aimed to create a more complex game with embedded mini games within the action cards. We divided tasks early on, with Jolomi focusing on making the game visually appealing and working on the design components, while Ruhani worked on building the core game logic. However, as we progressed, we found this division didn't hold up. We ended up creating a list of all the essential features and elements we wanted in the game, and from there, we took turns tackling different tasks.

One major obstacle was that our coding styles differed significantly, leading to clashes between our code when integrating different parts. This resulted in constant debugging every time a new component was added, slowing down our progress. Debugging became one of the most time-consuming parts of our work. Eventually, we had to simplify our original game concept, removing mini-games and focusing on making the core board game functional and fun. We experimented with ideas like integrating a snake-style mini-game but decided against it after realizing it would require more lines of code than the main game itself. In the end, we kept action cards with different types — point cards, automated movement cards, and more — to keep gameplay interesting without overcomplicating development. Despite the setbacks, we managed to finish with a working, enjoyable game.

6. Timeline of Activities

Week	Tasks Completed	Programmer
Week 1	Defined core variables: player count, player positions, spinner result, scores, and animations. Created the canvas and basic UI buttons for player count and spin action.	Ruhani
Week 2	Created the game board and background elements, including a sandy-colored board and light blue background.	Jolomi
Week 3	Designed and implemented player characters and their starting positions. Debugged crashing issues that occurred during the board setup.	Ruhani
Week 4	Developed the spinner mechanics and successfully linked them to player movement across the board.	Ruhani
Week 5	Implemented smooth player animations for movement transitions on the board.	Jolomi
Week 6	Designed and integrated action cards, including visual design and applying game effects when drawn.	Jolomi
Week 7	Collaborated to brainstorm and finalize additional improvements for smoother gameplay and better user experience.	Ruhani & Jolomi
Week 8	Conducted final playtesting, identified bugs, and polished animations and game flow.	Ruhani & Jolomi

7. Results

a. How to Play BoardRush

- Setup: Choose 2 to 4 players, each with a unique colour, and everyone starts with 100 points.
- Gameplay: On your turn, spin the spinner to move along the board. As you land on squares, keep an eye out for “STOP.” These trigger either action or point cards.
- Action Cards tell you to move extra spaces, switch positions or even take another turn.
- Point Cards adjust your score—some boost your points or double them, while others might knock points off. Your score never goes below zero.

- **Finishing:** When you reach the final “FINISH” square, you’re done. Finish order matters: the first player to finish earns 400 bonus points, the second 300, then 200, and 100 for last.

b. Results

We have made a fully functional board game where:

- **The Game Logic is Functional:** The players should be able to play a full game against each other. They will spin the spinner and work through the action cards to gain points. The player with most points at the end will win.
- **Advancement through Action Cards:** Every time an action card pops up, there will be a fun activity the players can play and earn points from.
- **Valid Gameplay and Win Condition:** The game will ensure that the game ends when all players reach the end of the board. Then, the statistics will be shown and there will be a celebratory page once the game is complete with an option to restart.
- **User Interface:** A simple and good-looking game experience

By implementing this project, we have demonstrated our understanding of what we have learned in our DMII class.

8. Conclusions and Future Work

We ended up creating a list of all the essential features and elements we wanted in the game, and from there, we took turns tackling different tasks. One major obstacle was that our coding styles differed significantly, leading to clashes between our code when integrating different parts. This resulted in constant debugging every time a new component was added, slowing down our progress. Eventually, we had to simplify our original concept, removing mini-games and focusing on making the core board game functional and fun. We experimented with ideas like integrating a snake-style mini-game but decided against it after realizing it would require more lines of code than the main game itself. In the end, we kept action cards with different types — point cards, automated movement cards, and more — to keep gameplay interesting without overcomplicating development. Despite the setbacks, we managed to finish with a working, enjoyable game. In the future we could work on fully expanding on our original idea of having mini-games within our board game and fully achieving what was in our original plan

9. External Materials

a. Development Tools:

- Programming Language: JavaScript
- Program: P5js, used as the website for us to make our code.

b. Hardware:

- A computer/laptop for each team member with Python installed.
- Access to the internet for research and collaboration.

c. Additional Learning Resources

- We used learning resources like YouTube as our current knowledge may not be sufficient for this project.

10. Appendix (Provide the code developed for the project)

```

const squareSize = 60; // square size

let numPlayers = 0; // number of
players

let currentPlayer = 0; // index of the
current player

let spinnerResult = 0; // spinner result

let playerPositions = [0, 0, 0, 0]; //
player positions

let playerOffsets = [
  { x: 45, y: 45 },
  { x: 15, y: 15 },
  { x: 15, y: 45 },
  { x: 45, y: 15 },
]; // offsets for players

let actionCardVisible = false; // card
visibility (action or point)

let spinnerDisplay = ""; // spinner
display

let currentActionCard = ""; // current
card text (action or point)

let currentActionEffect = ""; // for
action cards, store effect details

let currentCardType = ""; // "action" or
"point"

let currentPointValue = 0; // holds the
point value for point cards

let animating = false; // track if
animation is in progress

let animationStep = 0; // current
animation step

let totalSteps = 0; // total steps to
move based on spinner result

let lastMoveFrame = 0; // track last
move frame

const animationSpeed = 15; // delay
between moves

let remainingSteps = 0; // steps
remaining after landing on a stop tile

let switchButtons = []; // buttons for
switch action

let gameOver = false; // flag for game
over

const squares = [
  { x: 370, y: 50 }, { x: 310, y: 50 }, { x:
250, y: 50 }, { x: 190, y: 50 },
  { x: 130, y: 50 }, { x: 70, y: 50 }, { x: 10,
y: 50 }, { x: 10, y: 110 },
  { x: 10, y: 170 }, { x: 70, y: 170 }, { x:
130, y: 170 }, { x: 190, y: 170 },
  { x: 250, y: 170 }, { x: 310, y: 170 }, { x:
370, y: 170 }, { x: 370, y: 230 },
  { x: 370, y: 290 }, { x: 310, y: 290 }, { x:
250, y: 290 }, { x: 190, y: 290 },
  { x: 130, y: 290 }, { x: 70, y: 290 }, { x:
10, y: 290 }
];

const stopSquares = [6, 12, 18]; //
positions that trigger a card

const playerColors = ["blue", "red",
"yellow", "green"];

let playerScores = [100, 100, 100,
100]; // players start with 100 points

```

```

const actionCards = [
  "Move forward 2 spaces.",
  "Switch places with another player.",
  "Go back to start.",
  "Take another turn.",
  "Move backward 1 space.",
];

let finishedPlayers = []; // Track indices
of finished players

function setup() {
  createCanvas(450, 400);

  // Removed single player option.

  let button2 = createButton("2
  Players").mousePressed(() =>
  setPlayers(2)).position(90, 360);

  let button3 = createButton("3
  Players").mousePressed(() =>
  setPlayers(3)).position(170, 360);

  let button4 = createButton("4
  Players").mousePressed(() =>
  setPlayers(4)).position(250, 360);

  window.playerButtons = [button2,
  button3, button4];

  window.spinButton =
  createButton("Spin").mousePressed(s
  pinSpinner).position(375, 360);

  window.spinButton.hide();

  let scoresButton =
  createButton("Scores").mousePresse
  d(showScores).position(375, 20);
}

function draw() {
  if (gameOver) {
    drawGameOverScreen();
    return;
  }

  background("skyblue");
  drawBoard();

  // Animate players.
  for (let i = 0; i < numPlayers; i++) {
    let pos = playerPositions[i];

    // Only animate the current player if
    not finished.
    if (animating && i === currentPlayer)
    {
      let stepsLeft = totalSteps -
      animationStep;

      if (stepsLeft > 0 && frameCount -
      lastMoveFrame >= animationSpeed) {
        playerPositions[i] = Math.min(pos
        + 1, squares.length - 1);

        animationStep++;

        lastMoveFrame = frameCount;

        let { x, y } = squares[pos];

        let { x: offsetX, y: offsetY } =
        playerOffsets[i];

        fill(playerColors[i]);

        ellipse(x + offsetX, y + offsetY, 20,
        20);
      }

      // Display spinner result.
      if (spinnerDisplay !== "") {
        fill("black");
        textSize(32);
      }
    }
  }
}

```



```

    text(spinnerDisplay, width / 2 - 20,
height / 2);
}

```

```

// Display card (action or point) with a
close "X" button.

```

```

if (actionCardVisible) {
    fill(255);

    rect(150, 100, 150, 150);

    fill(0);
    textSize(16);

    text("Instructions", 180, 120);

    textSize(14);

    text(currentActionCard, 160, 150,
130, 90);
}

```

```

// Draw close "X" button.

```

```

fill(200);

rect(150 + 150 - 25, 100 + 5, 20, 20);

fill(0);

textSize(16);

text("X", 150 + 150 - 20, 100 + 20);
}

```

```

// Indicate current player's turn by
color.

```

```

fill("black");

textSize(16);

text(` Turn:
${playerColors[currentPlayer]} `, 20,
370);
}

```

```

function drawGameOverScreen() {

    background("lightgreen");

    fill("black");
}

```

```

textSize(28);

textAlign(CENTER, CENTER);

text("Congratulations!", width / 2, 50);

textSize(20);

text("Leaderboard", width / 2, 90);

```

```

let leaderboard = [];

for (let i = 0; i < numPlayers; i++) {

    leaderboard.push({ color:
playerColors[i], score: playerScores[i]
});

}

leaderboard.sort((a, b) => b.score -
a.score);

```

```

textSize(18);

let startY = 130;

for (let i = 0; i < leaderboard.length;
i++) {

    let entry = leaderboard[i];

    text(`${i + 1}. ${entry.color} -
${entry.score} points`, width / 2,
startY + i * 30);

}

```

```

textSize(16);

text("Press SPACE to restart", width /
2, height - 30);

}

```

```

function
handlePlayerFinish(playerIndex) {

    if
(!finishedPlayers.includes(playerIndex
)) {

        finishedPlayers.push(playerIndex);

        let finishOrder =
finishedPlayers.length;
}
}

```

```

let finishPoints = [400, 300, 200,
100][finishOrder - 1] || 0;

playerScores[playerIndex] +=
finishPoints;
}

```

```

if (finishedPlayers.length ===
numPlayers) {

    gameOver = true;

    window.spinButton.hide();

} else {

    advanceTurn();

}

}

```

```

function advanceTurn() {

    actionCardVisible = false;

    window.spinButton.show();

    do {

        currentPlayer = (currentPlayer + 1) %
numPlayers;

    } while
(finishedPlayers.includes(currentPlay
er));
}

```

```

if (finishedPlayers.length ===
numPlayers) {

    window.spinButton.hide();

}

}

```

```

function setPlayers(players) {

    numPlayers = players;

    playerPositions.fill(0, 0, numPlayers);

    playerScores = new
Array(numPlayers).fill(100);

    finishedPlayers = [];
}

```

```

gameOver = false;

window.playerButtons.forEach(button
=> button.hide());

window.spinButton.show();
}

function spinSpinner() {
  spinnerResult = Math.floor(random(1,
7));

  spinnerDisplay = spinnerResult;

  totalSteps = spinnerResult;

  animating = true;
  animationStep = 0;

  lastMoveFrame = frameCount;

  setTimeout(() => spinnerDisplay = "",
1000);
}

function drawBoard() {
  strokeWeight(2);

  fill(246, 215, 176);

  stroke(62, 103, 214);

  squares.forEach((square) => {
    rect(square.x, square.y, squareSize,
squareSize);
  });

  fill("black");

  textSize(16);

  stopSquares.forEach(index => {
    let square = squares[index];

    text("STOP", square.x + 10, square.y
+ 30);

  });

  textSize(16);

  text("START", 375, 85);

  text("FINISH", 15, 325);
}

function showScores() {
  let scoresMessage = "Scores:\n";

  for (let i = 0; i < numPlayers; i++) {
    scoresMessage +=
` ${playerColors[i]}: ${playerScores[i]}
points\n`;
  }

  alert(scoresMessage);
}

function showCard() {
  if (random(1) < 0.5) {
    showPointCard();
  } else {
    showActionCard();
  }
}

function showActionCard() {
  currentCardType = "action";

  currentActionCard =
actionCards[Math.floor(random(actio
nCards.length))];

  currentActionEffect =
currentActionCard;

  actionCardVisible = true;

  window.spinButton.hide();
}

function createSwitchButtons() {
  switchButtons.forEach(button =>
button.remove());

  switchButtons = [];
}

if
(currentActionEffect.includes("Switch
places with another player")) {
  createSwitchButtons();
}

function showPointCard() {
  currentCardType = "point";

  const pointCards = [50, 70, -30, -50,
"double", "half"];

  let card =
pointCards[Math.floor(random(pointC
ards.length))];

  if (card === "double") {
    currentActionCard = "Point card:
Double your points.";
  } else if (card === "half") {
    currentActionCard = "Point card:
Halve your points.";
  } else if (card >= 0) {
    currentActionCard = "Point card:
Gain " + card + " points.";
  } else {
    currentActionCard = "Point card:
Lose " + Math.abs(card) + " points.";
  }

  currentPointValue = card;

  actionCardVisible = true;

  window.spinButton.hide();
}

```

```

let buttonY = 220;

for (let i = 0; i < numPlayers; i++) {

  if (i !== currentPlayer &&
  !finishedPlayers.includes(i)) {

    let button = createButton(` Switch
with ${playerColors[i]} `);

    button.position(160, buttonY);

    button.mousePressed(() => {

      [playerPositions[currentPlayer],
playerPositions[i]] =

      [playerPositions[i],
playerPositions[currentPlayer]];

      switchButtons.forEach(b =>
b.remove());

      switchButtons = [];

      if (remainingSteps > 0) {

        resumeMovement();

      } else {

        advanceTurn();

      }

    });

    switchButtons.push(button);

    buttonY += 30;

  }

}

let stayButton = createButton("Don't
Switch");

stayButton.position(160, buttonY);

stayButton.mousePressed(() => {

  switchButtons.forEach(b =>
b.remove());

  switchButtons = [];

  if (remainingSteps > 0) {

    resumeMovement();

  } else {

    advanceTurn();

  });

  switchButtons.push(stayButton);

}

function handleActionEffect() {

  if
(currentActionEffect.includes("Move
forward 2 spaces")) {

    playerPositions[currentPlayer] =
Math.min(playerPositions[currentPlay
er] + 2, squares.length - 1);

  } else if
(currentActionEffect.includes("Move
backward 1 space")) {

    playerPositions[currentPlayer] =
Math.max(playerPositions[currentPlay
er] - 1, 0);

  } else if
(currentActionEffect.includes("Go
back to start")) {

    playerPositions[currentPlayer] = 0;

  } else if
(currentActionEffect.includes("Take
another turn")) {

    spinSpinner();

    return;

  }

  if (remainingSteps > 0) {

    resumeMovement();

  } else {

    advanceTurn();

  }

}

function handlePointCard() {

  if (currentPointValue === "double") {

    playerScores[currentPlayer] *= 2;

  } else if (currentPointValue === "half")
{

    playerScores[currentPlayer] =
Math.floor(playerScores[currentPlayer
] / 2);

  } else {

    playerScores[currentPlayer] +=
currentPointValue;

    if (playerScores[currentPlayer] < 0) {

      playerScores[currentPlayer] = 0;

    }

    if (remainingSteps > 0) {

      resumeMovement();

    } else {

      advanceTurn();

    }

  }

}

function resumeMovement() {

  animating = true;

  totalSteps = remainingSteps;

  animationStep = 0;

  lastMoveFrame = frameCount;

  remainingSteps = 0;

}

function mousePressed() {

  if (actionCardVisible) {

    let cardX = 150, cardY = 100,
cardWidth = 150, cardHeight = 150;

    let closeX = cardX + cardWidth - 25;

    let closeY = cardY + 5;

    let closeWidth = 20, closeHeight =
20;

```

```

    if (mouseX >= closeX && mouseX <=
closeX + closeWidth &&
        mouseY >= closeY && mouseY <=
closeY + closeHeight) {
        dismissCard();
    }
}

function dismissCard() {
    actionCardVisible = false;

    switchButtons.forEach(button =>
button.remove());

    switchButtons = [];

    if (currentCardType === "point") {
        handlePointCard();
    } else {
        if
        (currentActionEffect.includes("Switch
places with another player")) {
            if (remainingSteps > 0) {
                resumeMovement();
            } else {
                advanceTurn();
            }
        } else {
            handleActionEffect();
        }
    }

    // Restart the game when spacebar is
    pressed at game over.

    function keyPressed() {
        if (gameOver && key === ' ') {
            restartGame();
        }
    }

    function restartGame() {
        // Reset game variables while keeping
        the same number of players.

        textAlign(LEFT, CENTER);

        playerPositions = new
        Array(numPlayers).fill(0);

        playerScores = new
        Array(numPlayers).fill(100);

        finishedPlayers = [];

        currentPlayer = 0;

        gameOver = false;

        animating = false;

        actionCardVisible = false;

        remainingSteps = 0;

        window.spinButton.show();
    }
}

```